

AIN-2002

Artificial Intelligence Engineering Department

Bahçeşehir University

Dr. Binnur Kurt

Spring 2026

LEARNING ANALYTICS: DATA PREPARATION, CLASSIFICATION, CLUSTERING AND PCA

3rd Homework Assignment

Due on: May 31st, 2026

Introduction

This assignment focuses on an integrated data science workflow in Python. You will work with a synthetic learning analytics dataset that includes engagement metrics, assessment scores, teaching methods, and success status. The dataset intentionally contains missing values, inconsistent categorical labels, duplicated student identifiers, invalid e-mail formats, and logically impossible numerical values.

Your objective is to convert the raw dataset into an analysis-ready form and then apply exploratory analysis, principal component analysis, supervised classification, unsupervised clustering. Your submission must be reproducible and must include concise methodological interpretations.

Learning Objectives

- Detect and handle missing values using defensible imputation strategies.
- Normalize inconsistent categorical variables using string operations and mapping logic.
- Identify duplicate records, invalid identifiers, and logically impossible outliers.
- Create derived variables using binning, dummy variables, and case-based transformation.
- Use regular expressions to validate and extract information from text fields.
- Apply PCA to standardized numerical variables and interpret explained variance and loadings.
- Build a binary classification model and evaluate it using a confusion matrix and performance metrics.
- Apply K-Means clustering, select a reasonable number of clusters, and interpret cluster profiles.
- Report technical findings using clear, reproducible, and evidence-based reasoning.

Dataset

Use the following Python code to create the assignment dataset. Do not manually edit the generated DataFrame before beginning the tasks. All cleaning and transformation steps must be performed through Python code.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(2026)
n = 90

student_ids = np.arange(3001, 3001 + n)

df_hw3 = pd.DataFrame({
    "Student_ID": student_ids,
    "Department": rng.choice(
        ["AIE", "aie ", "Artificial Intelligence Engineering",
        "SE", "software engineering", "CE", "Computer Eng."],
        size=n,
        replace=True,
        p=[0.30, 0.05, 0.05, 0.25, 0.05, 0.25, 0.05]
    ),
    "Study_Hours": np.round(rng.normal(loc=13, scale=4, size=n), 1),
    "Sleep_Hours": np.round(rng.normal(loc=7, scale=1.2, size=n), 1),
    "Attendance_Rate": np.round(rng.uniform(low=0.45, high=1.00, size=n), 2),
    "LMS_Clicks": rng.poisson(lam=120, size=n),
    "Video_Minutes": np.round(rng.normal(loc=260, scale=80, size=n), 0),
    "Forum_Posts": rng.poisson(lam=4, size=n),
    "Teaching_Method": rng.choice(
        ["Traditional", "Flipped", "Project-Based"],
        size=n,
        replace=True,
        p=[0.35, 0.35, 0.30]
    ),
})
```

```

"Quiz_Score": np.round(rng.normal(loc=72, scale=12, size=n), 0),
"Project_Score": np.round(rng.normal(loc=78, scale=10, size=n), 0),
"Advisor_Email": [f"student{i}@bahcesehir.edu.tr" for i in student_ids]
})

# Introduce missing values
df_hw3.loc[[2, 17, 46], "Study_Hours"] = np.nan
df_hw3.loc[[10, 51], "Sleep_Hours"] = np.nan
df_hw3.loc[[6, 32], "Attendance_Rate"] = np.nan
df_hw3.loc[20, "Quiz_Score"] = np.nan
df_hw3.loc[43, "Project_Score"] = np.nan

# Introduce inconsistent labels and invalid strings
df_hw3.loc[[4, 24, 43], "Department"] = ["aie ", "SE ", "computer eng."]
df_hw3.loc[[8, 38, 69], "Teaching_Method"] = ["flipped ", "PROJECT-based", "traditional"]
df_hw3.loc[[11, 35], "Advisor_Email"] = [
    "student3012[at]bahcesehir.edu.tr", "student3036@invalid"
]

# Introduce logically impossible or extreme values
df_hw3.loc[79, "Student_ID"] = df_hw3.loc[9, "Student_ID"]
df_hw3.loc[15, "Attendance_Rate"] = 1.25
df_hw3.loc[27, "Study_Hours"] = -4
df_hw3.loc[40, "Sleep_Hours"] = 18
df_hw3.loc[63, "Quiz_Score"] = 132
df_hw3.loc[72, "Project_Score"] = -15
df_hw3.loc[54, "Video_Minutes"] = 980

# Create outcome variables before cleaning so that you must recompute them later
df_hw3["Overall_Score"] = np.round(
    0.40 * df_hw3["Quiz_Score"] +
    0.40 * df_hw3["Project_Score"] +
    0.20 * (100 * df_hw3["Attendance_Rate"]), 0
)
df_hw3["Success_Status"] = np.where(
    df_hw3["Overall_Score"] >= 75, "Successful", "At_Risk"
)

# Add two duplicated records
df_hw3 = pd.concat([df_hw3, df_hw3.loc[[5, 13]]], ignore_index=True)

```

Dataset Description

The dataset contains synthetic student-level learning analytics records. The table below summarizes the variables and the intentional data quality issues embedded in the dataset.

Column	Description	Expected Type	Data Quality Issues
Student_ID	Unique student identifier.	integer	Contains duplicated identifiers.
Department	Student department/program label.	categorical/category	Contains inconsistent spelling, case, and spacing.
Study_Hours	Weekly study hours.	numeric	Contains missing values and an impossible negative value.
Sleep_Hours	Average daily sleep hours.	numeric	Contains missing values and an implausible extreme value.
Attendance_Rate	Attendance represented as a proportion from 0 to 1.	numeric	Contains missing values and a value greater than 1.
LMS_Clicks	Number of LMS interactions.	numeric	No intentional missing value.
Video_Minutes	Total course video minutes watched.	numeric	Contains an extreme high value.
Forum_Posts	Number of forum contributions.	numeric	No intentional missing value.
Teaching_Method	Teaching method used in the course section.	categorical/category	Contains inconsistent case and spacing.
Quiz_Score	Quiz score on a 0-100 scale.	numeric	Contains missing value and score above 100.
Project_Score	Project score on a 0-100 scale.	numeric	Contains missing value and negative score.
Advisor_Email	Advisor/student contact-style e-mail string.	character	Contains invalid e-mail-like strings.
Overall_Score	Weighted score derived from quiz, project, and attendance.	numeric	Must be recomputed after cleaning.
Success_Status	Binary outcome derived from Overall_Score.	categorical/category	Must be recomputed after cleaning.

Assignment Overview

Complete the following analytical tasks in Python. Your response should include Python code, tables, visualizations, model output, and short written interpretations.

Section	Topics Covered	Expected Output
Task 1	Setup and data inspection	Structure, summary statistics, missing-value report
Task 2	Data cleaning and preparation	Clean dataset and justified cleaning decisions
Task 3	Feature engineering and transformation	Derived variables, bins, dummies, and regex validation
Task 4	Exploratory data analysis	Histogram, boxplot, bar chart, and scatter plot
Task 5	Principal Component Analysis	Explained variance, loadings, and PC plot
Task 6	Classification modeling	Binary classifier for Success_Status
Task 7	Classification performance	Confusion matrix and performance metrics
Task 8	K-Means clustering	Cluster interpretation

Tasks

Task 1 - Setup and Data Inspection

- Load the required Python libraries. You may use pandas, numpy, matplotlib, seaborn, scikit-learn, scipy, statsmodels, and standard Python functions.
- Create the dataset exactly using the Python code provided above.
- Print the first six rows, inspect the structure using `df.info()`, and produce summary statistics using `df.describe()` for all relevant variables.
- Report the number of missing values in each column and identify duplicated `Student_ID` values.
- List the unique values of `Department` and `Teaching_Method` before cleaning.

Task 2 - Data Cleaning and Preparation

- Normalize `Department` and `Teaching_Method` using trimming, case normalization, and mapping rules. Convert both variables into clean categorical variables.
- Detect and remove exact duplicated records. For duplicated `Student_ID` values, keep one analytically defensible record and justify your decision.
- Identify logically impossible values: `Study_Hours` < 0, `Sleep_Hours` outside a realistic range, `Attendance_Rate` outside [0, 1], `Quiz_Score` outside [0, 100], `Project_Score` outside [0, 100], and `Video_Minutes` extreme values.
- Correct invalid values by converting them to NA and then applying median imputation for continuous variables. Briefly justify why the median is preferred in the presence of outliers.
- After cleaning, verify that all numeric variables are within valid analytical ranges.

Task 3 - Feature Engineering and Transformation

- Recompute `Overall_Score` using the same weighted formula after cleaning `Quiz_Score`, `Project_Score`, and `Attendance_Rate`.
- Create `Success_Status` as a two-level categorical variable: 'Successful' if `Overall_Score` >= 75 and 'At_Risk' otherwise.
- Create `Performance_Level` using `np.select()`, pandas conditional logic, or an equivalent Python approach: 'High' if `Overall_Score` >= 85, 'Medium' if `Overall_Score` >= 70, and 'Low' otherwise.
- Create an `Engagement_Band` variable by binning `LMS_Clicks` or `Video_Minutes` into at least three ordered categories using `pandas.cut()`.
- Create dummy/indicator variables for `Department` or `Teaching_Method` using `pandas.get_dummies()` or another appropriate Python approach.
- Validate `Advisor_Email` using a regular expression. Report the invalid e-mail strings and extract the numeric student identifier from valid e-mail addresses.

Task 4 - Exploratory Data Analysis (EDA)

- Visualize the distribution of `Overall_Score` using a histogram.
- Compare `Overall_Score` across `Teaching_Method` using a boxplot.
- Create a bar chart showing the number of Successful and At_Risk students by Department.
- Create a scatter plot of `Study_Hours` versus `Overall_Score` and color the points by `Performance_Level`.
- Interpret each visualization in one or two sentences. Focus on distribution shape, group differences, and possible anomalies.

Task 5 - Principal Component Analysis (PCA)

- Select numerical learning-behavior variables such as `Study_Hours`, `Sleep_Hours`, `Attendance_Rate`, `LMS_Clicks`, `Video_Minutes`, `Forum_Posts`, `Quiz_Score`, and `Project_Score`.
- Standardize the selected variables and apply PCA using `sklearn.preprocessing.StandardScaler` and `sklearn.decomposition.PCA`.
- Report the explained variance ratio for each principal component and the cumulative variance preserved by the first two principal components.
- Inspect the loadings of PC1 and PC2 and interpret which original variables contribute most strongly to each component.

- Create a two-dimensional PC1-PC2 scatter plot and color points by Success_Status or Department.

Task 6 - Classification Modeling

- Split the cleaned dataset into training and testing sets using a 70/30 split with random_state = 2026.
- Build a logistic regression classifier using sklearn.linear_model.LogisticRegression or statsmodels Logit to predict Success_Status.
- Use meaningful predictors such as Study_Hours, Attendance_Rate, LMS_Clicks, Video_Minutes, Forum_Posts, Quiz_Score, Project_Score, Department, and Teaching_Method.
- Report the model summary and identify predictors that appear important for classification.
- Generate predicted probabilities for the test set and convert them into predicted classes using a threshold of 0.50.

Task 7 - Classification Performance Evaluation

- Construct a confusion matrix comparing predicted classes with actual Success_Status values in the test set.
- Manually compute accuracy, precision, recall/sensitivity, specificity, and F1-score. Clearly state which class is treated as the positive class.
- Interpret the confusion matrix. Discuss whether the model is better at identifying Successful students or At_Risk students.
- Briefly discuss one limitation of evaluating the model using a single random train-test split.

Task 8 - K-Means Clustering

- Use standardized numerical behavior variables, excluding Overall_Score and Success_Status, to perform clustering.
- Evaluate k values from 2 to 6 using total within-cluster sum of squares and silhouette score.
- Select a reasonable number of clusters and run sklearn.cluster.KMeans with n_init = 25 and random_state = 2026.
- Report cluster sizes and cluster centers in the original analytical context.
- Visualize clusters using the first two principal components or two selected standardized variables.
- Compare the discovered clusters with Success_Status using a cross-tabulation and interpret whether the clusters have practical meaning.

Submission Format

- Submit a single Jupyter Notebook file named student-id-HW3.ipynb, replacing student-id with your actual student ID.
- Your file must include all Python code, outputs, visualizations, and written interpretations.
- You may also submit an exported PDF or HTML output, but the Jupyter Notebook source file is mandatory.
- All visualizations must be complete and properly labeled with titles, X-axis labels, Y-axis labels, and legends where relevant.
- All tables must have clear column names and should be readable without requiring the instructor to inspect your code.

Important Notes

- Your code must run from top to bottom without errors in a fresh Python environment or a restarted notebook kernel.
- Use clear and descriptive variable names.
- Every data-cleaning decision must be justified briefly. Unsupported deletion or arbitrary replacement will be considered incomplete.
- Graphs without titles or axis labels will be considered incomplete.
- Classification and clustering results must include interpretation, not only code output.
- Academic dishonesty includes but is not limited to copying code from classmates, submitting another student's work, or submitting AI-generated solutions as your own. Such behavior is subject to disciplinary action.
- Assignments submitted after the announced deadline will not be accepted unless an official extension has been granted.